

Plugin Developer Guide – Version 0.4 June 2026

This guide is for developers who want to implement AI evaluation plugins for the *Catalogue of AI Tests and Controls* in the *AI Assessment Sandbox Configurator* (see documentation in this Github repository: <https://github.com/lux-ai-factory/rfc>).

The goal is to explain the contract exposed to plugin authors, clarify the current extension points, and make it easier to identify which additional hooks would be useful.

1. Purpose

The system is built around a plugin model in which evaluation logic is implemented outside the core application and loaded dynamically at runtime.

A plugin author should be able to:

- define configuration as typed data
- parse datasets and models into domain-specific objects
- execute evaluation logic
- emit structured measurements
- report progress during long-running work
- optionally influence host-specific integration behavior through extra hooks

The core question behind this guide is not how to use the surrounding platform, but whether the plugin contract is expressive enough for real evaluation workloads.

2. Architecture Overview

At a high level, the architecture has four moving parts:

- a plugin project that contains one or more plugin classes
- an interface package that defines the base classes and shared models
- a plugin loader that discovers and imports plugin packages
- one or more host runtimes that instantiate plugins for discovery, validation, or execution

The same plugin can be loaded in more than one context. In the current reference implementation, one runtime imports plugins to inspect metadata and configuration, while another runtime imports them to execute evaluation jobs.

3. Discovery Model

Plugins are discovered from a configured root directory. The loader scans each top-level folder in that directory and looks for a Python package in one of these two layouts.

src layout	Direct Package layout
<pre> my-plugin-project/ ├── pyproject.toml └── src/ └── my_plugin/ ├── __init__.py └── plugin.py </pre>	<pre> my-plugin-project/ ├── pyproject.toml └── my_plugin/ ├── __init__.py └── plugin.py </pre>

The package must export the plugin class from `__init__.py`. The loader imports the package and registers every class that inherits from `BaseEvaluationPlugin`.

Important details:

- the exposed plugin name is the Python class name, for example `MyPlugin`
- the project folder name and package name do not have to match
- any import error during discovery prevents the plugin from being registered

4. Local Development Assumptions

The current reference setup expects a plugin root directory mounted into the runtimes that load plugins. A typical local arrangement looks like this:

```

/absolute/path/to/plugins/
└── my-plugin-project/

```

As long as the host runtime points its plugin search path at `/absolute/path/to/plugins`, the plugin can be discovered.

You do not need to understand the surrounding platform to implement a plugin, but you do need to know two practical constraints:

- your project must live under the configured plugin root
- both the discovery runtime and the execution runtime must be able to import the package

5. Creating a Plugin Project

Create a new project in your plugin workspace:

```
mkdir -p /absolute/path/to/plugins
cd /absolute/path/to/plugins
mkdir my-evaluation-plugin
cd my-evaluation-plugin
uv init --lib
uv add aisc-plugin-interface
```

Recommended structure:

```
my-evaluation-plugin/
├── pyproject.toml
├── README.md
├── src/
│   └── my_evaluation_plugin/
│       ├── __init__.py
│       └── plugin.py
└── uv.lock
```

NOTE: You can achieve the recommended structure automatically by using the CLI tool:

```
uv run aisc-plugin-interface init-plugin
```

This tool will:

- Prompt you for the plugin class name (default: MyPlugin)
- Ask for the relative path of the Python file that contains the plugin the same level as `__init__.py` (default: `plugin.py` at the same level as the root `__init__.py`)
- Update the root `__init__.py` with the appropriate plugin import.

6. Core Contract

Every plugin must inherit from `BaseEvaluationPlugin[T]`, where `T` is a Pydantic model representing plugin configuration schema.

The core contract is centered around input providers, evaluation core logic, and optional hooks.

- Inputs (DATASET or MODEL) are declared via the `@evaluation_input` decorator
- Access parsed input data in logic using `self.get_input_data("...")`
- `evaluate(config_data: dict)`: core execution logic
- `validate_config_form_data(config_form_data: dict)`: validates incoming configuration against your typed model
- `@metric("...")`: mark metric export methods

- `report_progress(TaskProgress(...))` (optional): progress reporting
- Artifacts and custom input providers can be added optionally

These pieces are sufficient for the basic plugin lifecycle:

1. receive configuration
2. parse input artifacts
3. execute evaluation logic
4. emit measurements

7. Minimal Example

This example reads a CSV dataset and computes two aggregate metrics.

[src/my_evaluation_plugin/plugin.py](#)

```
from typing import Any

from pydantic import BaseModel, Field
from aisc_plugin_interface import (
    BaseEvaluationPlugin, Measure, TaskProgress, metric, evaluation_input, InputType
)
from aisc_plugin_interface.input_providers.csv_input_provider import CsvInputProvider

class ConfigSchema(BaseModel):
    score_column: str = Field(
        ...,
        description="Name of the CSV column containing numeric scores.",
    )
    threshold: float = Field(
        default=0.5,
        ge=0.0,
        le=1.0,
        description="Scores greater than or equal to this value count as passing.",
    )

    @evaluation_input(
        name="eval-data", input_provider_class=CsvInputProvider, input_type=InputType.DATASET
    )

class ExampleCsvPlugin(BaseEvaluationPlugin[ConfigSchema]):
    def evaluate(self, config_data: dict) -> Any:
        config = self.validate_config_form_data(config_data)
```

```

rows = self.get_input_data("eval-data")

scores: list[float] = []
total_rows = len(rows)

for index, row in enumerate(rows):
    raw_value = row.get(config.score_column)
    if raw_value is None or raw_value == "":
        continue

    scores.append(float(raw_value))

self.report_progress(
    TaskProgress(
        progress=(index + 1) / total_rows,
        extra={"rows_processed": index + 1},
    )
)

n_passing = sum(score >= config.threshold for score in scores)

return {
    "average_score": (sum(scores) / len(scores)) if scores else 0.0,
    "pass_rate": (n_passing / len(scores)) if scores else 0.0,
}

@metric("Average score")
def average_score_metric(self, evaluation_output: dict) -> list[Measure]:
    return [
        Measure(
            name="Average score",
            score=float(evaluation_output["average_score"]),
        )
    ]

@metric("Pass rate")
def pass_rate_metric(self, evaluation_output: dict) -> list[Measure]:
    return [
        Measure(
            name="Pass rate",
            score=float(evaluation_output["pass_rate"]),
            unit="ratio",
        )
    ]

```

[src/my_evaluation_plugin/__init__.py](#)

```

from .plugin import ExampleCsvPlugin

__all__ = ["ExampleCsvPlugin"]

```

8. Isolating evaluations and Lazy imports

Since plugin evaluations may require specific Python dependencies or even exact versions, the sandbox addresses this by creating a dedicated virtual environment for each plugin package. This ensures the evaluation runs with the requirements specified by the user.

As aforementioned, the plugin package is loaded at least twice:

- Once in the backend (by the loader) during plugin discovery
- Once by the evaluation engine when the user starts an evaluation

To prevent dependency conflicts or missing dependencies (`ModuleNotFoundError`), the plugin's unique dependencies should only be imported during the evaluation step. Therefore, it's essential to use lazy imports within the `evaluate` method to avoid any issues during plugin discovery.

9. Configuration as Typed Data

Plugin configuration is defined as a Pydantic model. That gives you:

- explicit configuration structure
- validation rules close to the plugin implementation
- a single source of truth for default values and constraints

Example:

```
class ConfigSchema(BaseModel):  
    threshold: float = Field(default=0.5, ge=0.0, le=1.0)
```

Inside `evaluate`, validate incoming configuration before using it:

```
config = self.validate_config_form_data(config_data)
```

Even if the host application also validates configuration, the plugin should treat validated config as the boundary between transport data and business logic.

10. Input Parsing

The plugin base class separates how inputs are delivered from how they are consumed.

Input Declaration:

- Use the `@evaluation_input` class decorator to declare plugin inputs, with a unique str identifier as its name
- Specify the input type via `InputType`: `InputType.DATASET` OR `InputType.MODEL`
- Provide the input provider class, which inherits from `BaseInputProvider`

Input Consumption:

- Access parsed input data using `self.get_input_data(...)` with the input name
- The input provider handles any specific logic for parsing and delivering input data.

Built-in CSV provider

The interface package includes `CsvInputProvider`, which turns CSV bytes into `list[dict]`.

Custom input providers

If you need another format, create a subclass of `BaseInputProvider`.

```
from aisc_plugin_interface import BaseInputProvider

class JsonInputProvider(BaseInputProvider):
    def _read_data(self, file_content: bytes):
        import json
        return json.loads(file_content.decode("utf-8"))
```

This design keeps parsing logic out of `evaluate` and makes it easier to test input handling independently.

11. Metrics and Measurements

The `evaluate` method may return any intermediate object. That object is then passed to each method decorated with `@metric`.

Metric methods must return `list[Measure]`.

Measure contains:

- name
- description
- unit
- score
- time
- error
- dimensions

One plugin can export one metric or many metrics. This separation is useful because it lets the evaluation step produce a shared intermediate result and keeps metric extraction methods small and focused.

Example:

```
@metric("Accuracy")
def accuracy_metric(self, evaluation_output) -> list[Measure]:
    ...
```

```
@metric("F1")
def f1_metric(self, evaluation_output) -> list[Measure]:
    ...
```

12. Progress Reporting

Long-running plugins can report progress during evaluation.

Option 1:

Use:

```
self.report_progress(TaskProgress(progress=0.25, extra={"stage": "loading"}))
```

Rules:

- progress must be between 0.0 and 1.0
- extra may contain plugin-defined metadata
- progress reporting is optional

Do not override `_set_progress_callback`; that is managed by the execution runtime.

Option 2:

Wrap your main evaluation loop inside `self.progress_bar()` which is similar to how `tqdm` progress bar works.

13. Optional Integration Hooks

The core plugin model is small, but the base class also exposes several optional hooks that are integration-oriented rather than strictly evaluation-oriented.

These hooks are important because they reveal where the architecture already allows extension and where future hooks may be needed.

`on_config_change`

This hook receives incomplete or partially edited configuration and can return:

- updated config data
- updated schema
- updated UI schema

It is useful for dynamic configuration, conditional fields, and derived defaults.

From a pure plugin-architecture perspective, this hook can be read more generally as:

- a way to react to evolving configuration state
- a place to derive secondary configuration fields
- a place to resolve configuration against partially known inputs

[parse_config_from_dataset](#)

This hook attempts to infer configuration from the current dataset.

It is useful when configuration depends on data shape, column names, task type, or metadata inferred from the dataset.

From an extensibility perspective, this suggests a broader family of possible hooks:

- infer config from model artifacts
- infer config from project metadata
- infer config from external registries or schemas

[get_metric_visualizations](#)

This hook returns visualization metadata associated with exported metrics.

It is not part of the core evaluation algorithm, but it is currently the way a plugin can communicate preferred result structure to a host application.

If the goal is to keep the plugin contract implementation-focused, this hook can be treated as optional integration metadata rather than a required plugin concern.

[feature_flags](#)

This hook exposes plugin-specific capabilities to the host application.

At the moment it is mostly used for host-specific behavior, but the pattern is potentially useful for broader capability negotiation between plugin and runtime.

[display_icon](#)

This hook is entirely presentation-oriented. It is useful only if the host application wants plugins to contribute presentation metadata.

NOTE: it is possible to set the `display_icon` using the plugin class attribute `ui_icon`. Same for the plugin display name used in the webapp with `plugin_name`.

If the document should remain strictly focused on implementation concerns, this hook is peripheral.

The easiest way to test a plugin is to decouple the concerns.